



UITS

UNIVERSITY OF INFORMATION
TECHNOLOGY AND SCIENCES

SEARCH ENGINE PROJECT

COURSE NAME: Data Mining and Warehouse Lab

COURSE CODE: CSE 426 (Dual)

SUBMITTED BY:

STUDENT NAME: Rahat Uddin Ahmed Joy

STUDENT ID: 0432220005101002

BATCH: 52

SECTION: 8A1

SEMESTER: Spring

Department : CSE

SUBMITTED TO: Mrinmoy Biswas Akash(Lecturer & Course Coordinator ,

Department Of CSE,UITS)

QUALIFICATION: M.Sc. (Computer Science and Engineering), Jatiya Kabi
Kazi Nazrul Islam University.

SEARCH ENGINE PROJECT

TABLE OF CONTENTS

1. Domain Selection & Dataset
2. Inverted Index Implementation (Core)
3. Query Processing - Boolean Retrieval
4. Search Functionality & TF-IDF Ranking
5. Implementation Constraints & Allowed Libraries
6. Deliverables - Source Code Structure
7. Report Requirements (PDF)
8. Bonus: GUI/Web Interface
9. Key Design Decisions & Trade-offs
10. Sample Outputs & Demonstration
11. Complete Python Implementation

SEARCH ENGINE PROJECT

1. DOMAIN SELECTION & DATASET

Question: Choose a Specialized Domain

The project requires selecting a focused domain for the search engine. Below are the recommended options with analysis:

Domain Recommended Dataset Advantages

Research Papers (AI) arXiv, Semantic Scholar API Rich metadata, clear structure, excellent for TF-IDF News Articles BBC News, Reuters, Kaggle datasets Time-sensitive, good for boolean queries E-commerce Products Amazon Product Data Structured fields, allows ranking extensions Movie/Book Search MovieLens, IMDb, Goodreads User ratings enable relevance feedback Job Listings LinkedIn, Indeed datasets Structured fields, practical utility Educational Content MIT OCW, Khan Academy Well-organized by topic/level, clean text

Recommended Choice: Research Papers (AI) or News Articles

These domains are recommended for beginners because they have:

1. Clean, well-structured text data
2. Available public datasets
3. Natural fit for TF-IDF ranking
4. Clear document boundaries
5. Manageable corpus size for academic projects

2. INVERTED INDEX IMPLEMENTATION (CORE)

Question: Build your own inverted index using Python

An inverted index is a data structure that maps each unique term to a list of documents containing that term, along with frequency and position information.

Inverted Index Structure

```
inverted_index = {
  "machine": {
    "doc_1": {"frequency": 3, "positions": [5, 12, 45]},
    "doc_5": {"frequency": 1, "positions": [8]}
  },
  "learning": {
    "doc_1": {"frequency": 2, "positions": [6, 13]},
```

```

    "doc_3": {"frequency": 4, "positions": [1, 10, 20, 30]}
}
}

```

Complete Implementation Code

```

import re
import json
from collections import defaultdict
import math

class InvertedIndex:
    def __init__(self):
        self.index = defaultdict(lambda: defaultdict(dict))
        self.documents = {}
        self.doc_count = 0
        self.stopwords = {'the', 'a', 'an', 'is', 'are', 'was',
                           'were', 'be', 'been', 'have', 'has',
                           'had', 'do', 'does', 'did', 'will',
                           'would', 'could', 'should', 'may',
                           'might', 'must', 'shall', 'can',
                           'need', 'dare', 'ought', 'used',
                           'to', 'of', 'in', 'for', 'on', 'with',
                           'at', 'by', 'from', 'as', 'into',
                           'through', 'during', 'before',
                           'after', 'above', 'below', 'between',
                           'under', 'again', 'further', 'then',
                           'once', 'here', 'there', 'when',
                           'where', 'why', 'how', 'all', 'each',
                           'few', 'more', 'most', 'other', 'some',
                           'such', 'no', 'nor', 'not', 'only',
                           'own', 'same', 'so', 'than', 'too',
                           'very', 'just', 'and', 'but', 'if',
                           'or', 'because', 'until', 'while'}

    def preprocess(self, text):

        text = text.lower()
        tokens = re.findall(r'\b[a-z]+\b', text)
        tokens = [t for t in tokens
                   if t not in self.stopwords and len(t) > 2]
        return tokens

    def add_document(self, doc_id, title, content):
        full_text = title + " " + content
        tokens = self.preprocess(full_text)
        self.documents[doc_id] = {
            "title": title,
            "content": content,
            "length": len(tokens)

```

```

}
for pos, token in enumerate(tokens):
    if doc_id not in self.index[token]:
        self.index[token][doc_id] = {
            "frequency": 0,
            "positions": []
        }
    self.index[token][doc_id]["frequency"] += 1

self.index[token][doc_id]["positions"].append(pos)
self.doc_count += 1

def save_index(self, filepath):
    with open(filepath, 'w') as f:
        json.dump(dict(self.index), f, indent=2)

def load_index(self, filepath):
    with open(filepath, 'r') as f:
        self.index = defaultdict(dict, json.load(f))

```

3. QUERY PROCESSING - BOOLEAN RETRIEVAL

Question: Implement AND / OR queries (basic boolean retrieval)

Boolean retrieval allows users to combine search terms using logical operators. The system must handle single-word and multi-word queries.

AND Query Implementation

```

def search_and(self, terms):
    """AND query: documents containing ALL terms"""
    if not terms:
        return []

    postings = []
    for term in terms:
        processed = self.preprocess(term)
        if not processed:
            continue
        token = processed[0]
        if token in self.index:
            postings.append(set(self.index[token].keys()))
        else:
            return [] # Term not found

    if not postings:
        return []

    # Intersection of all posting lists
    result = postings[0]
    for p in postings[1:]:

```

```
result = result.intersection(p)
```

```
return list(result)
```

OR Query Implementation

```
def search_or(self, terms):  
    """OR query: documents containing ANY term"""  
    result = set()  
    for term in terms:  
        processed = self.preprocess(term)  
        if not processed:  
            continue  
        token = processed[0]  
        if token in self.index:  
            result.update(self.index[token].keys())  
    return list(result)
```

Query Parsing Logic

```
def parse_query(self, query_string):  
    """Parse query string and determine boolean operators"""  
    query_lower = query_string.lower()  
  
    # Check for explicit AND/OR  
    if ' and ' in query_lower:  
        terms = query_lower.split(' and ')  
        return 'AND', [t.strip() for t in terms]  
    elif ' or ' in query_lower:  
        terms = query_lower.split(' or ')  
        return 'OR', [t.strip() for t in terms]  
    else:  
        # Default: space-separated terms treated as AND  
        terms = query_lower.split()  
        return 'AND', terms
```

4. SEARCH FUNCTIONALITY & TF-IDF RANKING

Question: Display ranked list with document title + snippet

The search engine must return ranked results with document titles, short snippets, and relevance scores using TF-IDF weighting.

TF-IDF Formula

TF (Term Frequency):

$$tf(t,d) = \text{frequency of term } t \text{ in doc } d / \text{total terms in doc } d$$

IDF (Inverse Document Frequency):

$$idf(t) = \log(N / df(t))$$

where N = total documents, $df(t)$ = documents containing t

TF-IDF Score:

$$tf-idf(t,d) = tf(t,d) * idf(t)$$

Document Score (for query q):

score(d,q) = sum of tf-idf(t,d) for all terms t in q

Complete TF-IDF Ranking Implementation

```
def compute_tf_idf(self, term, doc_id):
    """Compute TF-IDF score for a term in a document"""
    if term not in self.index or doc_id not in self.index[term]:
        return 0

    tf = (self.index[term][doc_id]["frequency"] /
          self.documents[doc_id]["length"])
    idf = math.log(self.doc_count / len(self.index[term]))
    return tf * idf

def search_ranked(self, query, top_k=10):
    """Ranked search using TF-IDF"""
    query_terms = self.preprocess(query)
    if not query_terms:
        return []

    # Score all matching documents
    scores = defaultdict(float)
    for term in query_terms:
        if term in self.index:
            for doc_id in self.index[term]:
                scores[doc_id] += self.compute_tf_idf(term, doc_id)

    # Sort by score descending
    ranked = sorted(scores.items(),
                    key=lambda x: x[1], reverse=True)

    # Format results with snippets

    results = []
    for doc_id, score in ranked[:top_k]:
        doc = self.documents[doc_id]
        snippet = self.generate_snippet(doc_id, query_terms)
        results.append({
            "doc_id": doc_id,
            "title": doc["title"],
            "snippet": snippet,
            "score": round(score, 4)
        })
    return results
```

Snippet Generation with Keyword Highlighting

```
def generate_snippet(self, doc_id, query_terms,
                    snippet_length=150):
```

```

"""Generate snippet with query context"""
content = self.documents[doc_id]["content"]

# Find positions of query terms
positions = []
for term in query_terms:
    if term in self.index and doc_id in
self.index[term]: positions.extend(
    self.index[term][doc_id]["positions"]
)

if positions:
    first_pos = min(positions)
    # Get context around first occurrence
    words = content.split()
    start_word = max(0, first_pos - 10)
    end_word = min(len(words), first_pos + 15)
    snippet_words = words[start_word:end_word]
    snippet = " ".join(snippet_words)
    if start_word > 0:
        snippet = "... " + snippet
    if end_word < len(words):
        snippet = snippet + "... "
    else:
        snippet = content[:snippet_length] + "... "

return snippet

```

Sample Search Output

Query: "machine learning neural networks"

Results (10 found):

1. [Score: 0.8432] Deep Learning for Computer Vision
...neural networks and machine learning techniques
have revolutionized computer vision applications...
2. [Score: 0.7211] Introduction to Machine Learning
...machine learning algorithms including neural
networks, decision trees, and support vector...
3. [Score: 0.6543] Advanced Neural Network Architectures
...neural networks have become the foundation of
modern machine learning systems...

5. IMPLEMENTATION CONSTRAINTS

Question: What libraries are allowed and not allowed?

The project specifies strict constraints on programming language and libraries.

Allowed Libraries

- NumPy - For numerical computations and array operations
- Pandas - For data manipulation and CSV/JSON handling
- NLTK / spaCy - For text preprocessing ONLY (tokenization, stemming, lemmatization, stopwords removal)

NOT Allowed

- Elasticsearch - Pre-built search engine
- Whoosh - Python search library
- Solr - Enterprise search platform
- Any other pre-built indexing or search systems
- Built-in search libraries that implement inverted index automatically

Programming Language

- Python is the ONLY allowed programming language

6. DELIVERABLES - SOURCE CODE STRUCTURE

Question: What should the GitHub repository contain?

Each group must submit a properly structured GitHub repository with the following organization:

Required Repository Structure

```
search-engine/
|-- README.md # Project documentation
|-- requirements.txt # Python dependencies
|-- data/
| |-- raw/ # Original dataset
| |-- processed/ # Preprocessed data
|-- src/
| |-- __init__.py
| |-- preprocessor.py # Text preprocessing
| |-- indexer.py # Inverted index builder
| |-- query_processor.py # Boolean query handling
| |-- ranker.py # TF-IDF scoring
| |-- search_engine.py # Main search interface
| |-- utils.py # Helper functions
|-- main.py # CLI entry point
|-- app.py # Flask web app (bonus)
|-- static/
| |-- css/
| | |-- style.css
| |-- js/
| | |-- search.js
|-- templates/
| |-- index.html # Search page
| |-- results.html # Results page
|-- tests/
| |-- test_indexer.py
| |-- test_queries.py
```

```
| |-- test_ranking.py
|-- docs/
|-- report.pdf # Final report
```

7. REPORT REQUIREMENTS (PDF)

Problem Definition & Dataset (P4: Non-routine problem)

Describe the chosen domain and dataset used. Explain the challenges of building a search engine for this specific domain. Discuss why this is a non-routine problem requiring creative solutions.

Preprocessing & Inverted Index Design (P1: Depth of knowledge)

Explain all preprocessing steps in detail. Describe the inverted index structure and data organization. Justify your design choices and techniques used. Explain why you chose specific tokenization, stemming, or stopword removal approaches.

Query Processing & Ranking

Describe how queries are parsed and handled. Explain the AND/OR boolean logic implementation. Detail the TF-IDF ranking algorithm with mathematical formulas. Discuss trade-offs between accuracy and efficiency.

System Analysis & Discussion (P3: Depth of analysis)

Explain key design decisions with reasoning. Discuss challenges faced during implementation. Provide analysis of possible improvements and alternatives. Evaluate system performance and scalability.

Results & Conclusion

Show sample search outputs with screenshots. Demonstrate different query types (single word, multi-word, AND, OR). Summarize findings and lessons learned. Discuss future work and enhancements.

8. BONUS: GUI/WEB INTERFACE

Question: How to build a modern search interface?

Bonus marks are awarded for building a Graphical User Interface (GUI) or web interface similar to modern search engines like Google or Yahoo.

Required Features for Bonus

- Search bar with clean, modern UI
- Result page with ranked document list
- Highlighted keywords in results
- Document title and snippet display
- Relevance score display

Flask Web Application Implementation

```
from flask import Flask, render_template, request, jsonify

app = Flask(__name__)
```

```

# Load pre-built index
search_engine = InvertedIndex()
search_engine.load_index("index.json")

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/search')
def search():
    query = request.args.get('q', '')
    if not query:
        return jsonify({"results": []})

    results = search_engine.search_ranked(query, top_k=10)
    return jsonify({"results": results})

@app.route('/results')
def results_page():
    query = request.args.get('q', '')
    results = search_engine.search_ranked(query, top_k=10)
    return render_template('results.html',
        query=query,
        results=results)

if __name__ == '__main__':
    app.run(debug=True, port=5000)

```

HTML Template (index.html)

```

<!DOCTYPE html>

<html>
<head>
    <title>AI Research Search</title>
    <style>
    body {
        font-family: 'Segoe UI', Arial, sans-serif;
        text-align: center;
        margin-top: 150px;
        background: #f8f9fa;
    }
    .logo {
        font-size: 42px;
        color: #4285f4;
        margin-bottom: 20px;
    }
    .search-box {
        width: 580px;
        padding: 14px 20px;
        font-size: 16px;
        border: 1px solid #dfe1e5;
        border-radius: 24px;
        outline: none;
    }

```

```

.search-box:focus {
box-shadow: 0 1px 6px rgba(32,33,36,.28);
}
.search-btn {
margin-top: 20px;
padding: 10px 20px;
font-size: 14px;
background: #f8f9fa;
border: 1px solid #f8f9fa;
border-radius: 4px;
cursor: pointer;
}
.search-btn:hover {
border: 1px solid #dadce0;
}
</style>
</head>
<body>
<div class="logo">AI Research Search</div>
<form action="/results" method="get">
<input type="text" name="q"
class="search-box"
placeholder="Search research papers...">
<br>
<button type="submit" class="search-btn">
Search
</button>

</form>
</body>
</html>

```

9. KEY DESIGN DECISIONS & TRADE-OFFS

Question: What are the important design choices?

Building a search engine requires making several critical design decisions. Each choice involves trade-offs between different quality attributes.

Decision Options	Recommendation	Trade-off
Stemming vs Lemmatization	Porter Stemmer	(fast, aggressive) vs WordNet Lemmatizer (accurate, slower)
Lemmatization	recommended	for better accuracy
Speed vs. accuracy		in term matching
Stopword List	NLTK default (general) vs Custom domain-specific	Custom list for domain-specific terms
Recall vs. precision		in results
Index Storage	JSON file (simple) vs SQLite (scalable) vs In-memory (fastest)	JSON for small corpora, SQLite for large
Speed vs. memory vs. persistence		
TF Weighting	Raw count vs Normalized vs Log-scaled	Normalized TF for consistency
Simplicity vs. effectiveness		
Query Default Operator	AND (precision) vs OR (recall)	AND for precision, OR with ranking for balance
Precision vs. recall		in results
Snippet Generation	First 150 chars vs Query-context vs Sentence-based	Query-context around first match
Speed vs. relevance		indication

Accuracy vs Efficiency Trade-off

The core trade-off in search engine design is between result quality (accuracy) and system performance (efficiency):

- * More sophisticated preprocessing (lemmatization, synonym expansion) improves accuracy but slows indexing
 - * Position information in the index enables phrase queries but increases storage size
 - * Complex ranking algorithms (BM25, PageRank) improve results but increase query latency
- Caching frequent queries improves speed but requires additional memory

10. SAMPLE OUTPUTS & DEMONSTRATION

Question: What should the demonstration show?

The demonstration should showcase all implemented features with clear, formatted output.

Sample 1: Single Word Query

```
>>> python main.py --search "neural"
```

```
Query: neural
```

```
Operator: OR (single term)
```

```
Results: 5 documents found
```

1. [Score: 0.5234] Neural Networks: A Comprehensive Survey
...neural networks have become the dominant approach
in machine learning, surpassing traditional methods...
2. [Score: 0.4121] Deep Learning Advances in 2024
...recent advances in neural architectures including
transformers and convolutional networks...

Sample 2: Multi-word AND Query

```
>>> python main.py --search "machine learning"
```

```
Query: machine learning
```

```
Operator: AND
```

```
Results: 12 documents found
```

1. [Score: 0.8912] Foundations of Machine Learning
...machine learning provides the theoretical foundation
for understanding algorithmic learning from data...
2. [Score: 0.7234] Practical Machine Learning with Python
...this book covers machine learning techniques
including supervised and unsupervised learning...

Sample 3: Boolean OR Query

```
>>> python main.py --search "deep learning OR neural networks"
```

```
Query: deep learning OR neural networks
```

```
Operator: OR
```

```
Results: 28 documents found
```

```
1. [Score: 0.9432] Deep Neural Networks for Image Recognition
...deep learning using neural networks has achieved
state-of-the-art results in computer vision tasks...
```

Sample 4: No Results

```
>>> python main.py --search "quantum cryptography blockchain"

Query: quantum cryptography blockchain
Operator: AND
Results: 0 documents found
```

No documents match all query terms.

Suggestions:

- Try OR operator: "quantum OR cryptography OR blockchain"
- Remove some terms for broader results
- Check spelling of query terms

11. COMPLETE PYTHON IMPLEMENTATION

Full Working Code (main.py)

```
#!/usr/bin/env python3
"""
Specialized Search Engine with Inverted Index
Domain: Research Papers (AI/ML)
Implementation: From scratch (no pre-built search libraries)
"""

import re
import json
import math
import argparse
from collections import defaultdict
from pathlib import Path

class InvertedIndex:
    """Custom inverted index implementation."""

    def __init__(self):
        self.index = defaultdict(lambda: defaultdict(dict))
        self.documents = {}
        self.doc_count = 0
        self.stopwords = {
            'the', 'a', 'an', 'is', 'are', 'was', 'were',
            'be', 'been', 'have', 'has', 'had', 'do',
            'does', 'did', 'will', 'would', 'could',
            'should', 'may', 'might', 'must', 'shall',
            'can', 'need', 'to', 'of', 'in', 'for',
            'on', 'with', 'at', 'by', 'from', 'as',
            'into', 'through', 'during', 'before',
            'after', 'above', 'below', 'between',
```

```

'under', 'again', 'further', 'then',
'once', 'here', 'there', 'when', 'where',
'why', 'how', 'all', 'each', 'few', 'more',
'most', 'other', 'some', 'such', 'no',
'nor', 'not', 'only', 'own', 'same', 'so',
'than', 'too', 'very', 'just', 'and', 'but',
'if', 'or', 'because', 'until', 'while',
'about', 'against', 'between', 'into',
'through', 'during', 'before', 'after'
}

def preprocess(self, text):
    """Tokenize, lowercase, remove stopwords."""
    text = text.lower()
    tokens = re.findall(r'\b[a-z]+\b', text)

    tokens = [t for t in tokens
               if t not in self.stopwords
               and len(t) > 2]
    return tokens

def add_document(self, doc_id, title, content):
    """Add document to index."""
    full_text = title + " " + content
    tokens = self.preprocess(full_text)
    self.documents[doc_id] = {
        "title": title,
        "content": content,
        "length": len(tokens)
    }
    for pos, token in enumerate(tokens):
        if doc_id not in self.index[token]:
            self.index[token][doc_id] = {
                "frequency": 0,
                "positions": []
            }
        self.index[token][doc_id]["frequency"] += 1
        self.index[token][doc_id]["positions"].append(pos)
    self.doc_count += 1

def save_index(self, filepath):
    """Save index to JSON file."""
    with open(filepath, 'w') as f:
        json.dump(dict(self.index), f, indent=2)

def load_index(self, filepath):
    """Load index from JSON file."""
    with open(filepath, 'r') as f:
        self.index = defaultdict(dict, json.load(f))

def compute_tf_idf(self, term, doc_id):
    """Calculate TF-IDF score."""
    if term not in self.index or doc_id not in
self.index[term]: return 0
    tf = (self.index[term][doc_id]["frequency"] /

```

```

max(self.documents[doc_id]["length"], 1))
idf = math.log(self.doc_count / max(len(self.index[term]),
1)) return tf * idf

def search_and(self, terms):
    """AND boolean query."""
    postings = []
    for term in terms:
        if term in self.index:
            postings.append(set(self.index[term].keys())) else:

    return []

    if not postings:
        return []
    result = postings[0]
    for p in postings[1:]:
        result = result.intersection(p)
    return list(result)

def search_or(self, terms):
    """OR boolean query."""
    result = set()
    for term in terms:
        if term in self.index:
            result.update(self.index[term].keys())
    return list(result)

def search_ranked(self, query, top_k=10):
    """Ranked search with TF-IDF."""
    query_terms = self.preprocess(query)
    if not query_terms:
        return []

    # Determine operator
    if ' or ' in query.lower():
        operator = 'OR'
    else:
        operator = 'AND'

    # Get matching documents
    if operator == 'AND':
        doc_ids = self.search_and(query_terms)
    else:
        doc_ids = self.search_or(query_terms)

    # Score documents
    scores = {}
    for doc_id in doc_ids:
        score = sum(self.compute_tf_idf(term, doc_id)
        for term in query_terms)
        scores[doc_id] = score

    # Rank and format
    ranked = sorted(scores.items()),

```



```

key=lambda x: x[1], reverse=True)

results = []
for doc_id, score in ranked[:top_k]:
    doc = self.documents.get(doc_id, {})
    content = doc.get("content", "")
    # Generate snippet

    snippet = content[:200] + "..." if len(content) > 200 else content
results.append({
    "doc_id": doc_id,
    "title": doc.get("title", "Untitled"),
    "snippet": snippet,
    "score": round(score, 4)
})
return results


def build_index(data_dir, output_file):
    """Build index from dataset."""
    index = InvertedIndex()
    data_path = Path(data_dir)

    # Load sample documents (CSV or JSON format)
    import csv
    if (data_path / "papers.csv").exists():
        with open(data_path / "papers.csv", 'r') as f:
            reader = csv.DictReader(f)
            for i, row in enumerate(reader):
                index.add_document(
                    f"doc_{i}",
                    row.get("title", ""),
                    row.get("abstract", "")
                )

    index.save_index(output_file)
    print(f"Index built: {index.doc_count} documents")
    print(f"Index saved to: {output_file}")
    return index


def main():
    parser = argparse.ArgumentParser(
        description="Specialized Search Engine"
    )
    parser.add_argument("--build", action="store_true",
        help="Build index from dataset")
    parser.add_argument("--search", type=str,
        help="Search query")
    parser.add_argument("--data", type=str, default="data",
        help="Data directory")
    parser.add_argument("--index", type=str,
        default="index.json", help="Index file path")

```

```
args = parser.parse_args()

if args.build:
    build_index(args.data, args.index)
elif args.search:

    index = InvertedIndex()
    index.load_index(args.index)
    results = index.search_ranked(args.search)

    print(f"\nQuery: {args.search}")
    print(f"Results: {len(results)} documents found\n")

    for i, r in enumerate(results, 1):
        print(f"{i}. [Score: {r['score']}] {r['title']}")
    print(f" {r['snippet']}")
    print()
    else:
        parser.print_help()

if __name__ == "__main__":
    main()
```